Invoice-Guard — Design Documentation

Nov 7, 2026 · @Buddhika Amarasinghe

1. Overview and scope

Invoice-Guard turns each supplier invoice PDF into exactly one ERP posting. A model only reads the invoice; a deterministic rule engine decides whether it can post; anything uncertain goes to a person. It implements Phase 1 of the Invoice Validation Pipeline TSD v2 (Nov 4, 2026) on branch `feat/tsd-v2-phase1`.

Goals the design serves

- **No double posting.** Exactly one ERP document per invoice, even though Kafka delivers at least once.
- **No lost invoice.** Every message ends Posted, Rejected or Duplicate, with an audit trail.
- **Uncertain means human.** Any failed rule or field confidence below 0.85 goes to review, never straight to the ERP.
- **Fast, trustworthy review.** A reviewer clears a typical exception in under 2.5 minutes without leaving the screen.

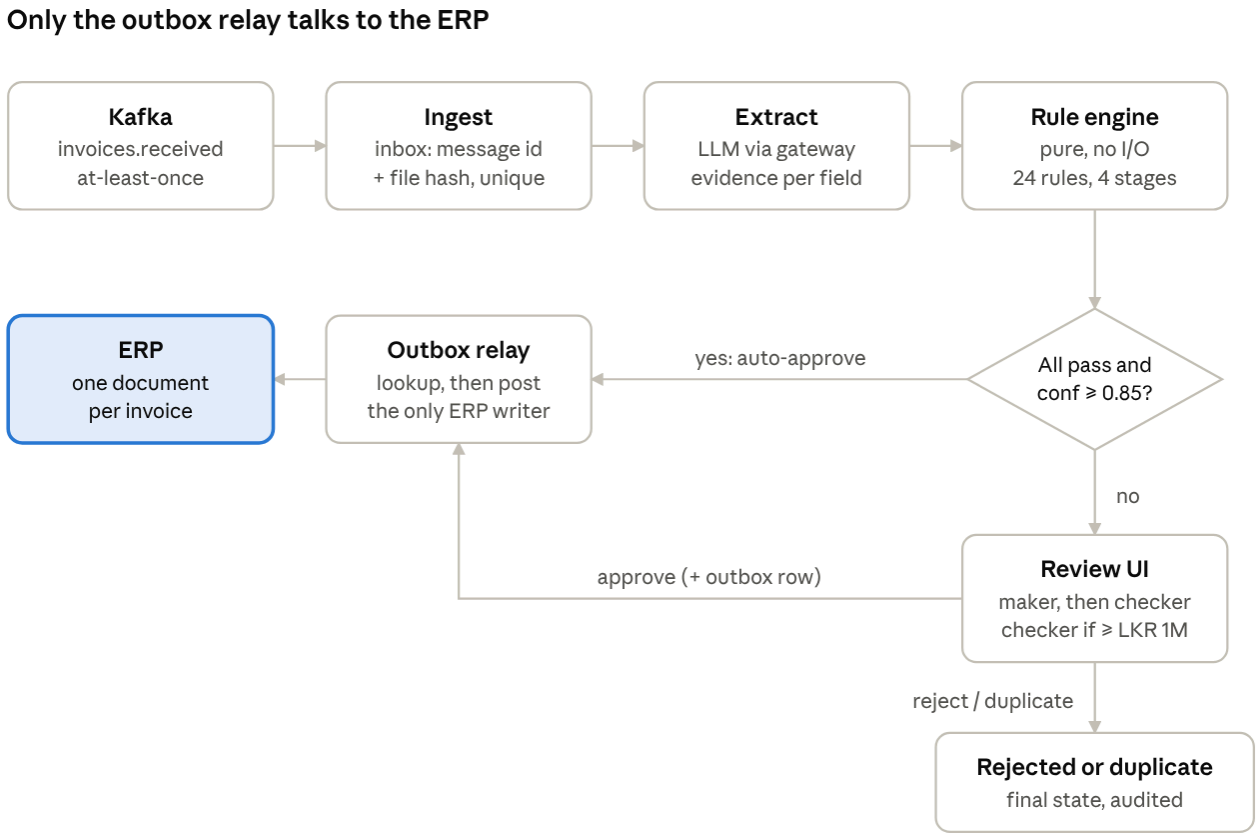
Phase 1 status

Area	State
Rule engine, state DB, inbox, outbox relay, review API, review UI, SLA escalation scheduler	Built and tested
Fake ERP with fault injection, demo seed, walkthrough	Built
SQLite (dev and pilot)	Tested
Kafka consumer, Anthropic extractor, Postgres	Wired, not yet exercised against real systems
SSO, reconciliation job, email/Teams delivery of SLA notifications	Not built (section 10)

Out of scope, as in the TSD: payment runs, supplier onboarding, PO creation, OCR model training and ERP-side approval workflows.

2. Architecture

The model only reads the invoice; deterministic code decides whether it posts, and the only way into the ERP is the outbox relay, reached by a clean auto-pass or a person's approval. Every step writes to the state DB before the next starts, so a crash resumes from the last committed step.



message-to-ERP flow · 7 components, 1 decision

A clean invoice goes straight from the rule engine to the relay; anything else waits for a person, and only an approval writes the outbox row the relay drains.

Components

Component	Process	Responsibility
Ingest	`services/ingest`	Kafka consumer; writes the inbox row, then commits the offset
Worker	`services/worker`	Leases RECEIVED invoices from the DB; extract, normalise, enrich, validate, route; sweeper for expired leases
LLM gateway	`libs/llm_gateway`	Structured outputs, shared token bucket, circuit breaker, repair re-prompt, fallback model

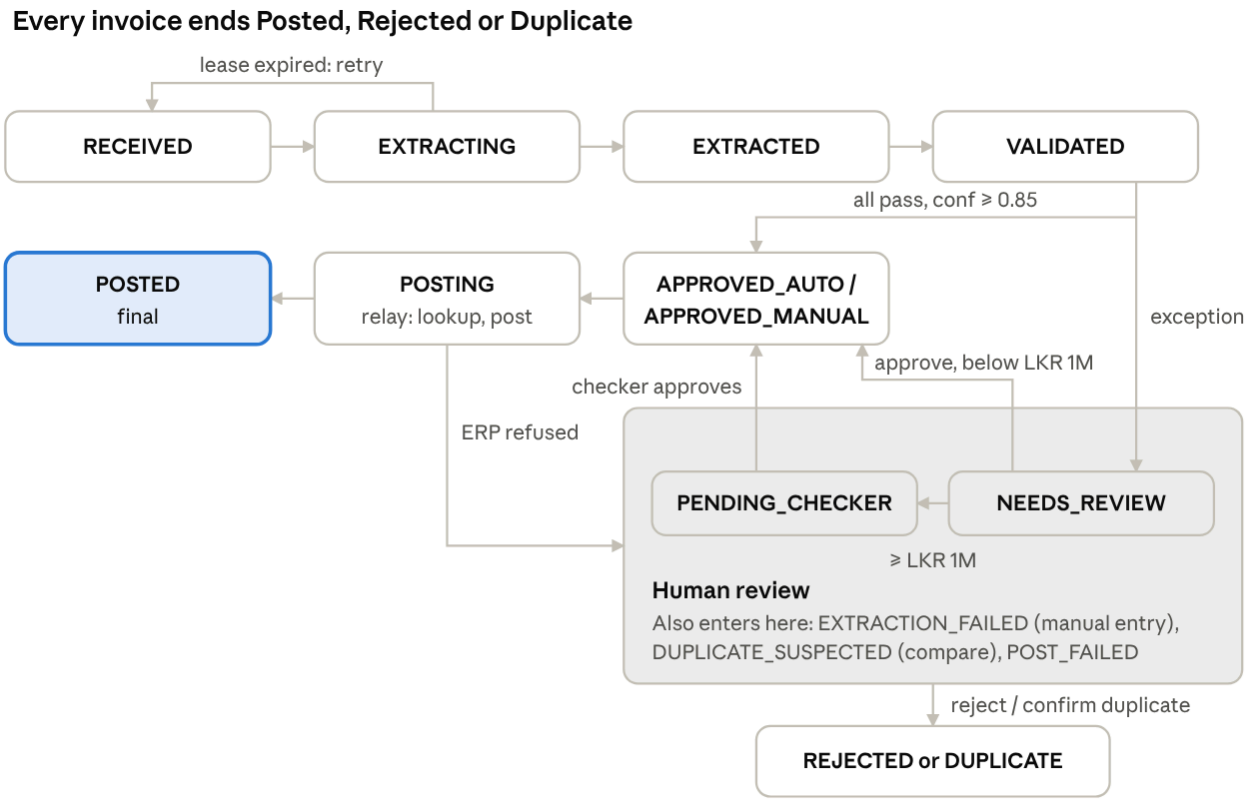
Component	Process	Responsibility
Rule engine	`libs/rules`	Pure, versioned validation; shared by worker and API
Review API	`services/api`	Queue, invoice, validate, decision, check, ops endpoints
Review UI	`apps/web`	Queue, workspace, checker view, ops dashboard
Outbox relay	`services/relay`	Drains the outbox; ERP lookup before every post
ERP adapter	`libs/erp_adapter`	Protocol with lookup by external reference; fake ERP for dev
Object store	`libs/core/store.py`	PDFs by SHA-256, write-once, signed 5-minute URLs
SLA scheduler	services/scheduler	Every 5 minutes: SLA clock in business hours (paused on hold), escalation once per level, priority recompute, notification delivery

Repository layout

```
apps/web/          Next.js review UI
services/ingest/   Kafka consumer -> inbox
services/worker/   extract, normalise, enrich, validate, route; sweeper
services/api/      review, validate, decision, check, ops endpoints
services/relay/    outbox -> ERP adapter (the only ERP writer)
services/scheduler/ SLA escalation every 5 minutes
libs/rules/        pure rule engine + versioned rule sets
libs/core/         normalisation, enrichment, routing, SLA, review workflow
libs/erp_adapter/  protocol + fake ERP with fault injection
libs/llm_gateway/  extraction schema, rate limiter, breaker
libs/db/           models, state machine, audit chain, migrations
scripts/           demo seed, walkthrough
```

3. Data model and state machine

The state DB is the source of truth for every invoice; Kafka is only the delivery channel and the ERP holds only the final result. Every state change is one transaction, guarded by a `version` column (optimistic locking), and writes one append-only, hash-chained event.



invoice state machine · main paths

The pipeline runs left to right along the top; anything it cannot decide drops into human review, and only an approval leads back up to posting. The table below lists every allowed transition (`libs/db/state.py`).

Invoice states

State	Meaning	Next
RECEIVED	Inbox written, PDF stored	EXTRACTING
EXTRACTING	Model call in progress (120 s lease)	EXTRACTED, EXTRACTION_FAILED, RECEIVED (retry)
EXTRACTED	Parsed and normalised	VALIDATED
EXTRACTION_FAILED	Retries exhausted; manual-entry task	NEEDS_REVIEW, REJECTED
VALIDATED	Rules evaluated	APPROVED_AUTO, NEEDS_REVIEW, DUPLICATE_SUSPECTED

State	Meaning	Next
DUPLICATE_SUSPECTED	Business key already live	DUPLICATE, NEEDS_REVIEW, REJECTED
NEEDS_REVIEW	In a person's queue	APPROVED_MANUAL, PENDING_CHECKER, REJECTED, DUPLICATE
PENDING_CHECKER	Maker done, at or above LKR 1M	APPROVED_MANUAL, NEEDS_REVIEW (sent back)
APPROVED_AUTO / APPROVED_MANUAL	Outbox row written	POSTING
POSTING	Relay calling the ERP	POSTED, POST_FAILED, back to APPROVED_* to wait
POST_FAILED	ERP refused (business error)	NEEDS_REVIEW
POSTED · REJECTED · DUPLICATE	Final	—

Core tables

Table	Holds	Unique constraints
`inbox`	message id, PDF hash, source, invoice id	`message_id` ; `pdf_sha256`
`invoice`	Header in minor units, state, version, confidence and evidence per field, business key, amount in LKR, rule-set version, posting period	`business_key` where state ≠ REJECTED (partial index)
`invoice_line`	Description, item code, UoM, PO line, qty (exact decimal text), unit price, amount, confidence, evidence	(`invoice_id` , `line_no`)
`extraction`	Attempt, model, prompt version, parsed output, latency, tokens, error	(`invoice_id` , `attempt`)

Table	Holds	Unique constraints
`rule_result`	One row per rule per evaluation, with expected, actual, message, fields	(`invoice_id` , `rule_key` , `evaluation_id`)
`review_task`	Kind, reasons, priority, assignee and lease, SLA due, hold (start, end, paused business seconds), escalation level and pool, maker, checker, corrections, overrides, results before and after	One open task per invoice (partial index)
`invoice_event`	Append-only audit: action, from/to state, actor, role, IP, payload, hash, previous hash	—
`outbox`	Idempotency key = invoice id, status (PENDING, UNKNOWN, DONE, FAILED), attempts, next attempt, ERP doc no.	`idempotency_key`
notification	SLA warning, breach and escalation messages: kind, recipients (user or role@entity), subject, body, status (PENDING, SENT, FAILED), attempts	(task_id, kind)

The same SQLAlchemy models run on SQLite (WAL, single writer) and Postgres 16; Alembic owns migrations. Timestamps go through a `UTCDateTime` type so they are always timezone-aware UTC on both databases.

4. Rule engine

The rule engine is one pure function, `evaluate(invoice, context, rule_set) → [RuleResult]`, with no I/O. The pipeline and the review API call the same function, so a reviewer always sees exactly what the agent saw. Being pure also makes it replayable for audits and unit-testable at the TSD's boundaries.

Evaluation order. Rules run in four stages. Every rule in every stage runs, so the reviewer sees all problems at once. A rule missing an input returns `SKIPPED` and names its dependency (for example `needs LINE_PO_MATCH`).

Stage	Rules	Card in the UI
1. Shape	DATE_PARSE, LINES_PRESENT	Date, Lines
2. Identity	SUPPLIER_RESOLVE, SUPPLIER_INACTIVE (BLOCK), DUP_EXACT (BLOCK), DUP_FUZZY, PO_OWNERSHIP	Supplier and duplicates
3. Arithmetic	LINE_ARITH, SUBTOTAL_SUM, TAX_CHECK, TOTAL_RECONCILE, TOTAL_POSITIVE	Total, Lines
4. Reference	DATE_NOT_FUTURE, DATE_AGE, DATE_AFTER_PO, DATE_PERIOD_OPEN, CURRENCY_MATCH, TOTAL_VS_PO, LINE_PO_MATCH, LINE_ITEM_MASTER, LINE_PRICE, LINE_UOM, LINE_QTY, LINE_REPEAT (WARN)	Date, Total, Lines

Thresholds live in a versioned `RuleSetConfig` per legal entity; each invoice records the version it was checked against. Defaults: 90 days age, 1-day future grace, ±2% price, +2% over remaining PO balance, ±1 minor unit on totals (overridable per currency), description similarity ≥ 0.90, fuzzy duplicates within 30 days and edit distance ≤ 2, confidence ≥ 0.85.

Money handling. Amounts are integers in minor units using ISO 4217 exponents (LKR and USD 2, JPY 0, KWD 3). Multiplication uses `Decimal` with `ROUND_HALF_UP`; floats never touch an amount. Line prices are compared per the PO's unit of measure, converted through the item master.

Routing. `decide()` returns one of three routes:

- `AUTO_APPROVE` — every rule passes, every required field ≥ 0.85, and (if configured) the amount is under the auto-post ceiling.
- `REVIEW` — with the reasons: failed or skipped rule codes plus `LOW_CONF:<field>`.
- `BLOCKED` — a BLOCK rule failed; a person can only reject or confirm a duplicate.

Approval gate. Approve is allowed only with no BLOCK, no SKIPPED, and a valid reason code on every remaining FAIL (`OTHER` needs a 20-character comment).

Queue priority orders the review queue (breached SLAs pinned first):

```
\text{priority} = 40\cdot\text{blocker} + 25\cdot\frac{\text{SLA used}}
{\text{SLA}} + 20\cdot\min\left(1,\frac{\text{amount}_{LKR}}{10^{7}}\right) +
15\cdot\text{duplicate suspicion}
```

5. Exactly-once posting and failure handling

Double posting is stopped at four layers, and the only path into the ERP is a transactional outbox whose relay looks the invoice up in the ERP before every post. The worst case for any failure is a delay or a person, never a second payment or a silent loss.

Duplicate layers

Layer	Catches	Mechanism
Message	Kafka redelivers the same message	`inbox.message_id` primary key; acknowledged as a no-op
File	Same PDF sent twice	`inbox.pdf_sha256` unique; object store is content-addressed and write-once
Business document	Same invoice re-scanned as a new file	Partial unique index on `business_key` (supplier, invoice no., date, total, currency) where state ≠ REJECTED
ERP edge	Crash after the ERP accepted, before our DB update	Relay calls `find_by_external_ref(invoice_id)` before every post

A colliding business key is stored, not rejected: the duplicate keeps its own invoice row with `business_key = NULL` and `duplicate_of` set, and goes to `DUPLICATE_SUSPECTED` for a person to compare. Dropping it would be the lost-invoice failure.

Why an outbox. Approving and "remember to post" must be one atomic fact. The approval transition and the `outbox` row commit in one transaction; the relay drains the outbox and is the only component allowed to call the ERP.

Crash and error handling

Failure	What happens	Result
Crash before the inbox write	Offset not committed; Kafka redelivers	No loss
Crash after the inbox write	Redelivery hits the unique key; acknowledged	No duplicate
Worker dies mid-extraction	120 s lease expires; sweeper returns it to RECEIVED	Retried; extra tokens at worst

Failure	What happens	Result
Model output fails the schema	One repair re-prompt, then the fallback model, then a manual-entry task	Never dropped
Quoted evidence not in the PDF text	Field confidence forced to 0	Field outlined for review
Provider 429 / 5xx / outage	Shared token bucket (10% under caps), backoff with full jitter up to 6 attempts, circuit breaker at 50% failures	Backlog waits in the DB
Relay dies after the ERP accepted	Outbox row still pending or UNKNOWN; next pass finds the document by external ref	One ERP document
ERP business error	POST_FAILED, then NEEDS_REVIEW with the ERP message	Person decides
ERP down	Back-off 1 min doubling to 30 min; invoice waits in APPROVED	No loss

The relay crash case is proven by an automated chaos test: the fake ERP accepts, then times out; the retry finds the document and `post_invoice` is called exactly once.

6. Review UI/UX

The UI is built around one number: reviewer time. At about 3 FTE per point of exception rate at month-end, every second saved per invoice matters, so the workspace puts evidence beside each value and makes the next action obvious.

Four screens

Screen	Who	What it does
Review queue	Reviewer	Tasks sorted breached-first, then by priority; SLA countdown pills and escalation chips (Warned, Supervisor pool, Finance mgr); reason chips; filters by entity, reason, SLA, amount band, supplier; "Next invoice" claims the top task for 15 minutes
Review workspace	Reviewer	PDF left (55%) with evidence boxes, validation cards right (45%), pinned action bar

Screen	Who	What it does
Checker view	Checker	Maker's decision, reason, comment, changed values (old struck through), overrides and rule results before → after; approve or send back only
Ops dashboard	Supervisor	Nine KPI tiles with drill-down; arrivals vs processed; queue depth by reason; reviewer throughput; backlog vs capacity

Design principles, and how each is built

- **Evidence beside the claim.** Clicking a field outlines its source box on the PDF; clicking a box focuses the field. Low-confidence boxes are outlined amber.
- **Exceptions first.** A list at the top of the panel orders problems the way people fix them: identity, date, total, lines. J and K step through it.
- **Live re-validation.** Each edit calls `/validate` after a 300 ms pause; nothing is saved. Changed results animate once and are announced through an `aria-live` region ("Total reconciles now passes").
- **Never colour alone.** Every status pairs colour, icon and word: ✓ Pass, ▲ Warn, ✕ Fail, – Not checked.
- **Numbers built for checking.** Tabular numerals, right-aligned, with the currency code and signed differences (`Diff +0.50`).
- **Keyboard first.** J/K issues, E edit, A approve, O override, R reject, D duplicate, H hold, N next invoice, ? help.

Workspace cards

- **Date card:** ISO date input with the long form below ("3 October 2026"), a "Confirm" button when day and month are ambiguous, and "Post in first open period" when the period is closed.
- **Total card:** a reconciliation strip, `Subtotal + Tax + Charges = computed · Invoice · Diff` , each term clickable to its evidence, with the remaining PO balance underneath.
- **Line items grid:** inline-editable cells; qty against open PO qty; unit price against PO price with variance %; a PO-line picker ranked by similarity; add, split and delete rows.
- **Action bar:** Approve (becomes "Send to checker" at LKR 1M), Approve with override, Reject, Mark duplicate, Hold – ask supplier, with the reason Approve is disabled always shown.

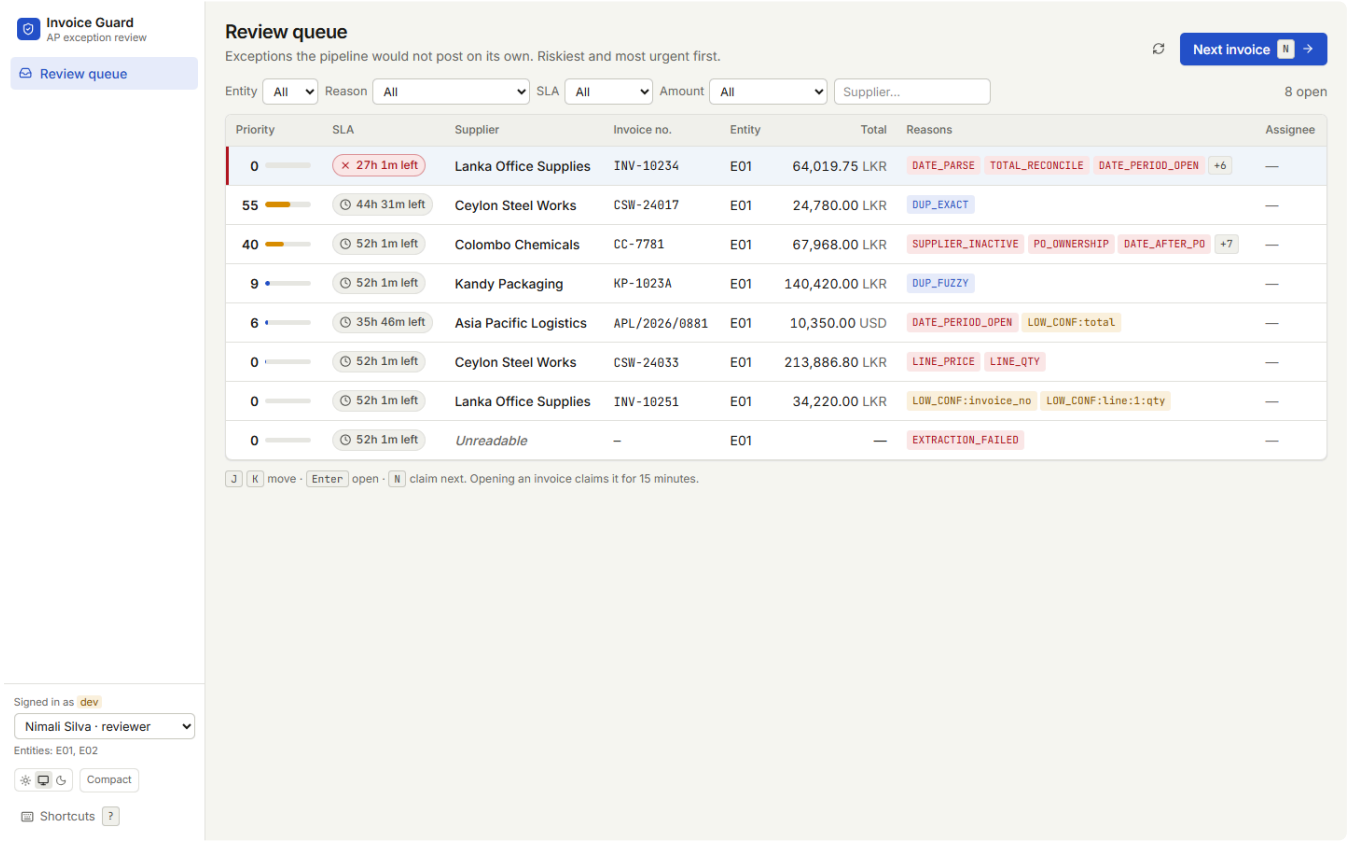
States every screen handles: loading skeletons; "Queue clear" with the next SLA risk; claimed by someone else (read-only, with their name); stale version (banner, draft kept); validate call failed (Approve disabled); ERP down (reference rules "not checked"); out-of-

scope link (404); narrow screens (read-only).

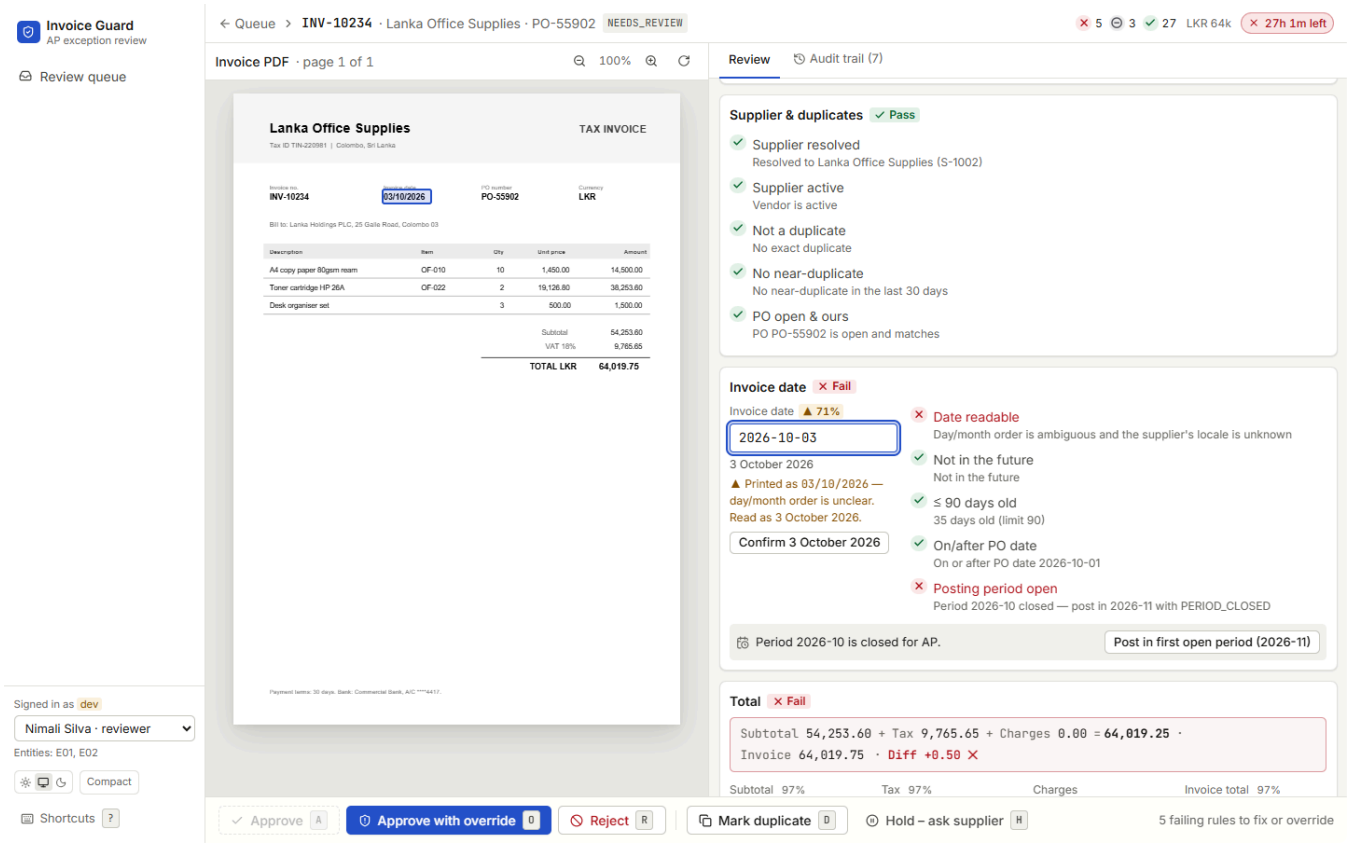
Visual system. Neutral surfaces with one accent for primary actions; status colours that hold 4.5:1 contrast on their tints; base text 14 px (13 px compact); light, dark and system themes. Chart colours were checked with a colour-blindness validator in both themes.

Screenshots (demo data, Chrome, 1440 × 900)

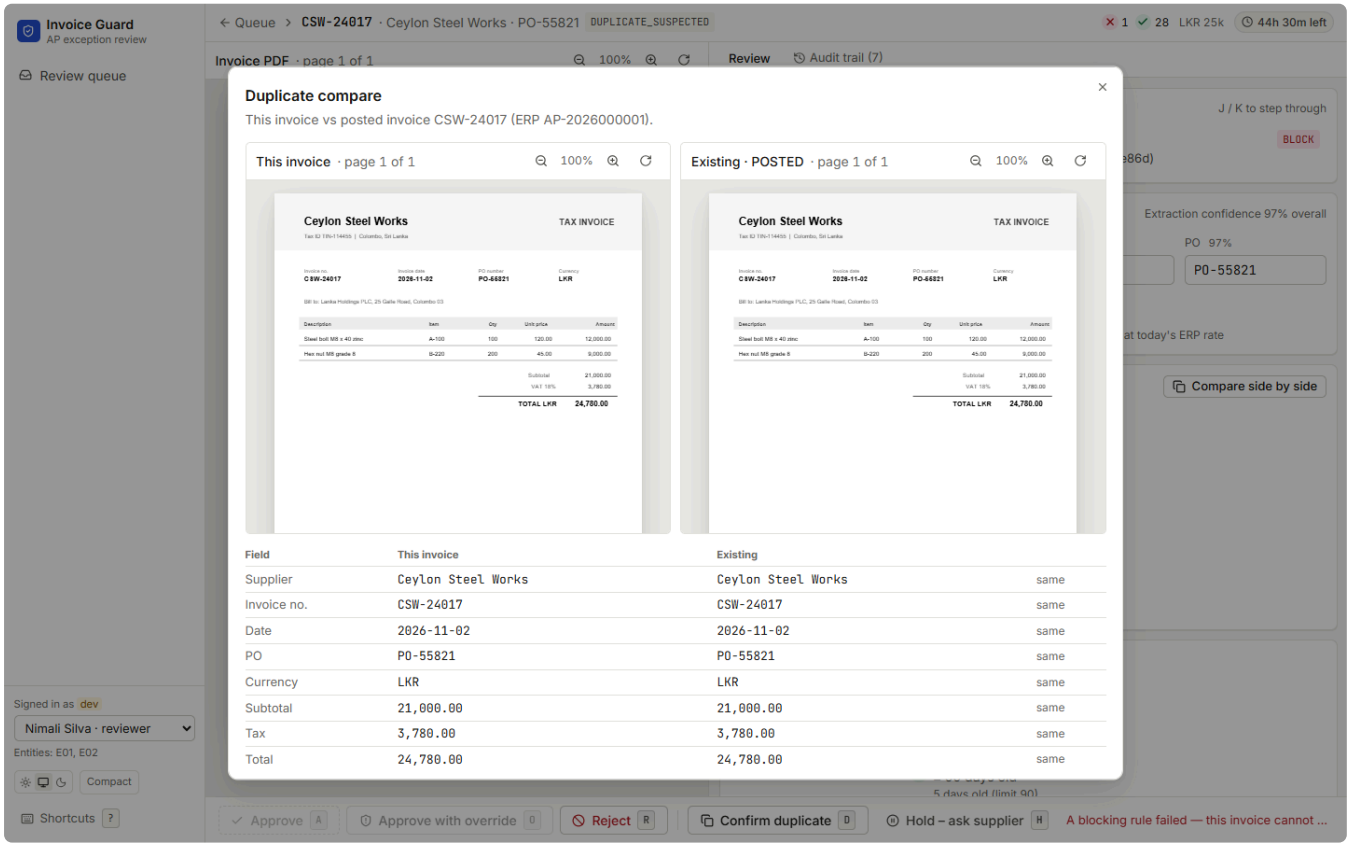
Review queue: breached SLAs pinned first, then by priority; reason chips say why each invoice needs a person.



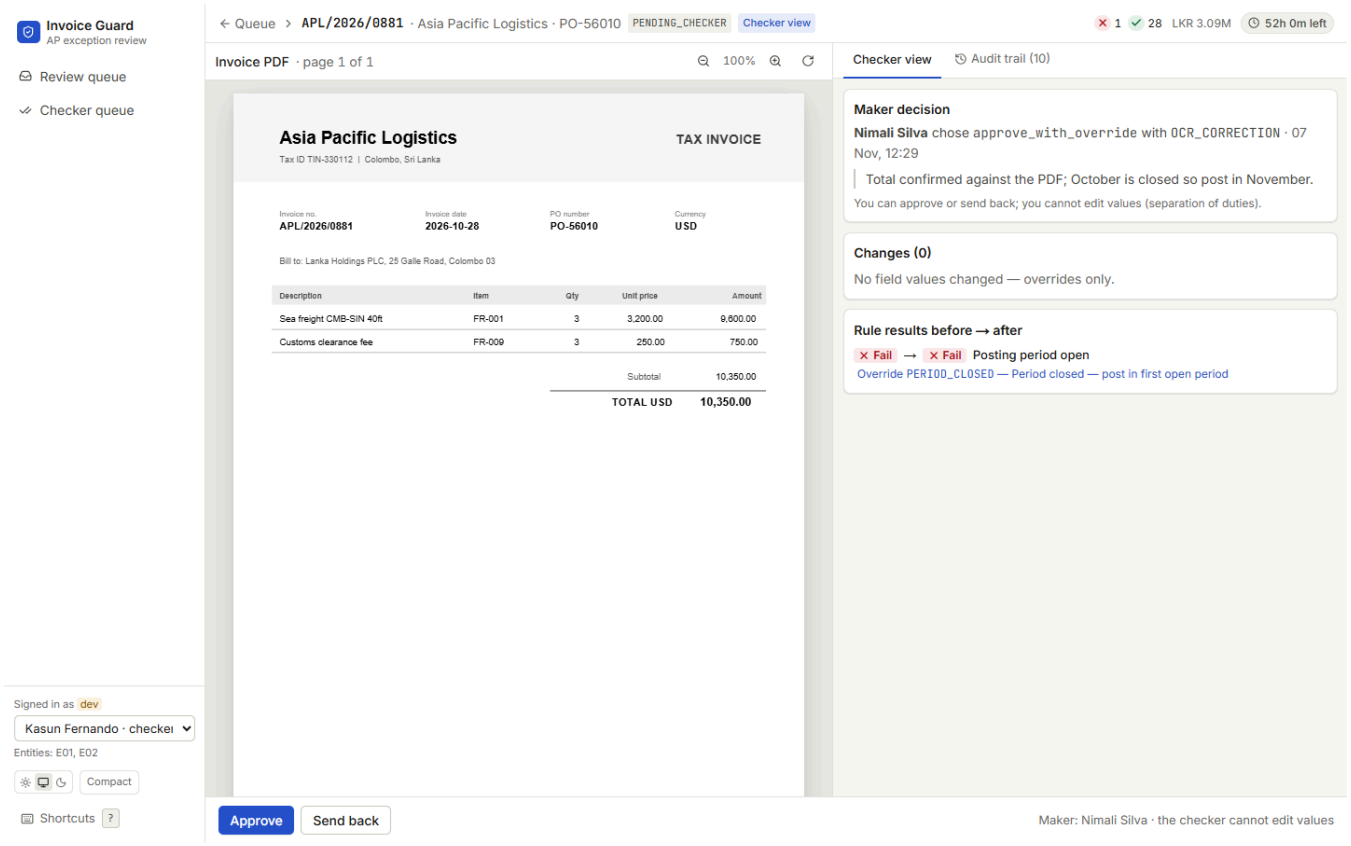
Review workspace on the TSD's example invoice: the selected date's source box is outlined on the PDF, every failing rule is listed, and Approve stays disabled.



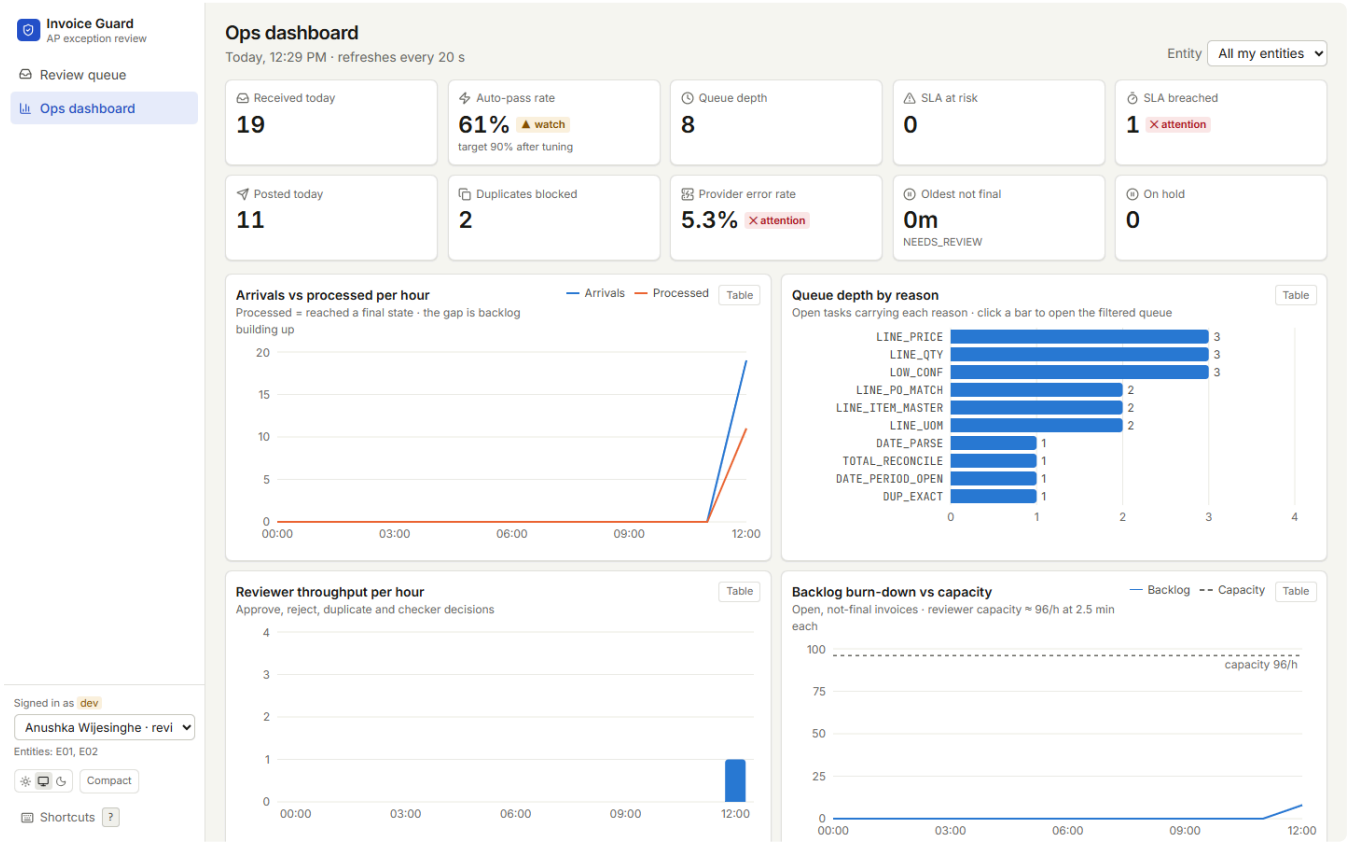
Duplicate compare: a re-scan of a posted invoice, both PDFs side by side with a field-by-field diff.



Checker view: the maker's reason, comment and override; the checker can only approve or send back.



Ops dashboard: KPI tiles with drill-down and four charts, each with a table view.



7. Human controls and security

Every control is enforced on the server in `libs/core/review.py`, not in the UI: a person can change what posts, but never alone at or above LKR 1M, never without a reason code, and never outside their legal entities.

Roles (scoped per legal entity)

Role	Can	Cannot
Reviewer (maker)	Claim, correct fields, override FAIL rules with a reason, approve below LKR 1M, reject, mark duplicate, hold	Approve own work at or above LKR 1M; override BLOCK rules
Checker	Approve or send back maker decisions at or above LKR 1M	Edit values; check an invoice they made (`checker_id ≠ maker_id`)
Supervisor	Everything a reviewer can, plus the ops dashboard and invoices escalated to the supervisor pool	Bypass maker-checker
Auditor	Read everything in scope, including the event log	Any write

Decision rules

- **Maker-checker:** a manual approval of an invoice at or above LKR 1M equivalent (ERP daily rate) goes to `PENDING_CHECKER` and creates a checker task. Send back needs a comment and returns the task to the same maker.
- **Reason codes** on every override, correction and rejection: PERIOD_CLOSED, PRICE_AGREED, QTY_PARTIAL, SUPPLIER_CONFIRMED, OCR_CORRECTION, DUPLICATE_CONFIRMED, NOT_OUR_INVOICE, OTHER (comment of 20+ characters).
- **Server re-validates on approval** and refuses with 409 if the rule results changed since the reviewer's last check (for example, the PO was consumed).
- **Claims and versions:** opening an invoice claims it for 15 minutes; a second reviewer gets 409 "Claimed by …". Decisions carry `If-Match: <version>`; a stale version is refused with 409 and the draft is kept.
- **Entity scope** is applied in the repository layer; an invoice outside the user's entities returns 404, so its existence is not revealed.

SLA and escalation. Review SLA is 8 business hours (4 in the month-end window: last 3 and first 2 business days), counted on the entity's calendar (08:30–17:30 Asia/Colombo, Mon–Fri). The clock pauses while a task is on hold; a hold lasts at most 3 business days and then the task resumes on its own. The queue shows on track, at risk (≥ 75%), breached (≥ 100%) and on hold.

A scheduler (`services/scheduler` , logic in `libs/core/escalation.py`) runs every 5 minutes. Each level fires once and never goes down, so running it twice, or after an outage, never double-notifies:

SLA used	Level	Action	Notified
75%	1 · Warned	Warning only	Assignee and team lead
100%	2 · Breached	Task moves to the supervisor pool: reviewers can't claim it (the current holder can finish); supervisors get it first from "Next invoice"	Supervisors and assignee
150% (125% at month-end)	3 · Second escalation	Stays with supervisors	Finance manager and supervisors

Every pass also banks expired holds, pushes `sla_due_at` out by time spent on hold, and recomputes queue priority so the order reflects time burning. Notifications are rows in `notification` , unique per task and level, delivered by a pluggable sender (a log sender in dev) with up to 5 retries. The queue shows Warned, Supervisor pool and Finance mgr chips; an escalated invoice shows a red banner.

Audit. Every action writes an append-only `invoice_event` with actor, role, IP, from/to state and payload (changed fields old → new, reason code, comment, rule results before and after). Events are hash-chained per invoice; `verify_chain` detects tampering.

Security. PDFs reach the browser only through HMAC-signed URLs that expire in 5 minutes, served with a content security policy that blocks scripts. The model has no tools and no posting rights; injection-like text in a PDF is flagged. Dev builds use an `X-User` header switcher in place of SSO (section 10).

Key API endpoints

Method + path	Purpose
`GET /queue?kind=&entity=&reason=&sla=&amount=`	Open tasks, breached first then priority

Method + path	Purpose
`POST /queue/next`	Claim the top task (15-minute lease)
`GET /invoices/{id}`	Header, lines, evidence, rule results, PO candidates, duplicate, task, events
`POST /invoices/{id}/validate`	Re-run the rules on draft corrections; nothing saved
`POST /invoices/{id}/decision`	approve, approve_with_override, reject, duplicate, not_duplicate, hold (`If-Match` required)
`POST /invoices/{id}/check`	Checker approve or send back
`GET /invoices/{id}/pdf`	Signed URL, 5-minute expiry
`GET /ops/metrics`	KPI tiles and chart series

8. Implementation and testing

The system is one Python monorepo with three background processes (ingest, worker, relay), a FastAPI review API and a Next.js app, verified by 79 automated tests, a demo seed that runs the real pipeline and a real-browser run of the UI.

Stack

Layer	Choice
UI	Next.js 15 (App Router), React 19, TypeScript, TanStack Query + Table, Tailwind 4, Radix dialogs, Recharts, PDF.js 5 (loaded unbundled)
API	FastAPI, Pydantic v2, SQLAlchemy 2, Alembic
Workers	Python asyncio; aiokafka for ingest
State DB	SQLite (WAL) in dev and pilot; Postgres 16 in production, same models
Extraction	Anthropic structured outputs (`messages.parse`); primary `claude-haiku-5-5` , fallback `claude-sonnet-5-5`
ERP	`ERPAdapter` protocol; fake ERP with fault injection for dev and tests

Tests (79, all passing)

Suite	Covers
Rule engine (53)	Every rule; TSD boundaries (exactly 90 days, ±1 minor unit, 2.0% vs 2.02%); UOM conversion; routing; approval gate; maker-checker; priority; Hypothesis property tests for money
Pipeline (7)	Auto-post; Kafka redelivery and same-file no-ops; relay crash chaos test; ERP down; re-scan duplicate never posts; unreadable PDF becomes a task; audit chain detects tampering
Review API (9)	Wireframe fix-override-approve posts in the open period; maker-checker separation; send back; entity-scope 404; stale version 409; claim conflict 409; BLOCK cannot be overridden; hold pauses the SLA and cannot stack; ops access
SLA scheduler (10)	Business-hour clock across nights and weekends; each level once and idempotent; missed levels raised after an outage; month-end 4 h SLA and 125% escalation; hold pauses then resumes with 27 h banked; priority rises with SLA used; supervisor pool and current-holder exception; delivery once; failing sender retries then gives up

Demo and walkthrough

- `scripts/seed_demo.py` generates 19 invoice PDFs with exact evidence boxes and runs them through ingest, worker, rules and relay: 11 auto-post, 7 go to review, 1 becomes a manual-entry task, plus one Kafka redelivery.
- `scripts/walkthrough.py` narrates one invoice end to end in the terminal: arrival, redelivery, extraction, rules, claim, correction, maker approval, checker approval, posting and the audit trail.
- CI runs ruff, mypy, pytest and an Alembic migration on a fresh DB, then ESLint, type checks and a production build of the web app.

9. Problems found while building, and their fixes

Eleven real defects surfaced from running the system rather than from reading the code: one in the rules, one in the model integration, five in the UI found by driving it in Chrome, three in the data layer, one in the demo data. All are fixed and committed.

Found by	Problem	Fix
Demo seed	`LINE_PRICE` compared a per-bundle price with the PO's per-each price and reported +2400%	Convert to the PO's unit of measure through the item master before comparing
Claude API check	Forced `tool_choice` returns 400 on current models	Structured outputs via `messages.parse(output_format=…)`
Browser run	Workspace crashed: Next's bundler re-bundling `pdfjs-dist` v5 throws `Object.defineProperty called on non-object`	Dropped `react-pdf`; PDF.js loads unbundled as a native ES module from `/public/pdfjs`
Browser run	Hydration mismatch on every page: identity lives in `localStorage`, which the server can't see	The app renders after mount
Browser run	A reviewer could not confirm an ambiguous date that was already right: no change meant no correction was sent	"Confirm 3 October 2026" button; ISO text input in place of the locale date picker, which showed `10/03/2026`
Screenshots	The page scrolled when a field got focus, pushing the header off screen: screen-reader-only text escaped the scrolling panel and stretched the document to 2224 px	Scroll containers are positioned and clipped; the page never scrolls
Screenshots	A card still said "Fail" after its only failure was overridden; a confirmed date still showed 71% confidence	"Pass · with override"; confirmed or corrected fields show 100%
SQLite	SQLite drops `tzinfo`, breaking lease and SLA comparisons	`UTCDateTime` column type: always aware UTC in Python
SQLAlchemy	Replacing invoice lines hit `UNIQUE(invoice_id, line_no)` because inserts flush before deletes	Flush the deletes before re-inserting

Found by	Problem	Fix
Migration review	The generated SLA migration added a NOT NULL column with no default (fails on any database with review tasks), and tasks already on hold would silently come off hold	Server default 0, plus a backfill that starts the pause for held tasks; upgrade tested on a populated database
Demo seed	The seed aged tasks by wall-clock hours, so on a weekend the business-hours clock showed almost no escalations	Seed ages tasks in business time to 80%, 110% and 160% of SLA, then runs one scheduler pass

10. Decisions, known gaps and next steps

Phase 1 proves the guarantees on SQLite with a fake ERP; the biggest remaining risks are unproven real-world integrations (SSO, Postgres, the model, a real ERP) and the missing reconciliation job.

Decisions taken where the TSD is open

- **DUP_EXACT is BLOCK.** A reviewer can confirm the duplicate or reject; "Not a duplicate" applies to near duplicates, or after the reviewer corrects the business key.
- **SKIPPED blocks approval** and cannot be overridden: you can't override a check that never ran.
- **Auto-post ceiling** (`auto_approve_ceiling_lkr_minor`) is a setting, off by default. It serves the pilot's LKR 250k limit and the TSD's open question on always reviewing invoices above LKR 10M.
- **Non-PO invoices and credit notes** stay exceptions (PO_OWNERSHIP and TOTAL_POSITIVE fail) until their own flows exist.

Known gaps

Gap	Impact	Next step
Dev header auth	Anyone can act as anyone in dev	OIDC (Entra ID or Okta), roles and entities from group claims
No email or Teams delivery	SLA warnings and escalations are recorded and logged, not delivered to inboxes	An SMTP or Microsoft Graph sender; resolve role@entity recipients from the directory

Gap	Impact	Next step
Postgres untested	`SKIP LOCKED` and partial unique indexes proven only on SQLite	Postgres in docker-compose and a CI job
Real extraction untested	Demo uses pre-recorded extractions	Run the 500-invoice eval set through the Anthropic extractor; gate on 98% field accuracy at ≥ 0.85
Fake ERP never consumes PO balances	Repeat invoices against one PO keep passing LINE_QTY	Track invoiced quantity, or read it from the real ERP
Rate limiter per process	Several workers could exceed provider limits	Redis-backed token bucket
Thresholds live in code	A change needs a deploy, not the TSD's two-admin approval	Versioned rule sets per entity in the DB
Not built	Daily reconciliation, OCR for scanned pages, OpenTelemetry, S3/Blob store, Playwright and axe-core in CI	Phases 2–3
No holiday calendar	SLA business days skip weekends only, so a public holiday burns SLA	Per-entity holiday list in the SLA calendar

Next steps, in order

1. Daily reconciliation job (the lost-invoice net) and real delivery for SLA notifications.
2. Postgres in CI, then OIDC SSO: prerequisites for the single-entity pilot.
3. Extraction evals on real invoices; tune the confidence threshold before the shadow run.
4. First real ERP adapter, confirming whether it supports native idempotency or only external-reference lookup.